

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2022

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, tháng 2 năm 2022

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022

IOI China candidate team papers / National training team 2022 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2022. Tập này là bản quét ảnh: pdftotext chỉ trả về các dấu sang trang và không khôi phục được văn bản thân bài. Vì vậy bản LaTeX này chuyển ngữ phân nhận diện tập sách, mục lục và bài đầu tiên bằng cách đọc trực tiếp các trang đã render từ PDF, kết hợp OCR bằng tesseract.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2022*. Trang bìa ghi đơn vị China Computer Federation và thời điểm phát hành là tháng 2 năm 2022. Tập PDF gồm 246 trang; phần thân bắt đầu từ bài về duy trì thông tin đường đi trên một lớp DAG đặc biệt.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt	Dai Jiangqi
13	DFT tổng quát hơn	Zhou Hangrui
28	Bàn về ứng dụng của một tư tưởng tham lam dựa trên so sánh trong thi tin học	Zhang Junkai
49	Bàn về các thuật toán liên quan tới đồ thị phẳng	Tang Shaoxuan
72	Bàn về bài toán tương tác dạng tìm tham số tương tác	Hu Hao
82	Bàn về thuật toán xấp xỉ trong OI	Xu Tingqiang
99	Khảo sát bước đầu về bài toán tô màu đồ thị	Peng Bo
118	Bàn về bài toán tối ưu không gian trong thi tin học	Chen Zhixuan
129	Bàn về một lớp ma trận Monge	Zhang Jingxing
145	Bàn về phân tích độ phức tạp của một lớp bảng băm	Chen Yusi
159	Ứng dụng của đại số tuyến tính trong OI	Chang Ruinian
182	Bàn về các bài toán tổ hợp chuỗi liên quan tới lý thuyết Lyndon	Wan Chengzhang
204	Báo cáo ra đề <i>Chuỗi của Cutie Duo</i>	Feng Shiyuan
213	Bàn về bài toán mô phỏng luồng chi phí trong OI	Chen Kuowen

3 Bàn về duy trì thông tin đường đi trên một lớp DAG đặc biệt

Dai Jiangqi, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Trong OI, không ít bài toán liên quan tới cấu trúc dữ liệu có thể được xem như bài toán đường đi trên DAG. Bài viết này giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(n \text{ poly}(\log S))$, trong đó n là kích thước đầu vào, S là tổng số đường đi của DAG và $\text{poly}(x)$ biểu thị một đa thức theo x . Bài viết cũng khảo sát ứng dụng của các thuật toán này trong cấu trúc dữ liệu, chuỗi và số học.

Mở đầu

Trong OI, rất nhiều bài toán có thể quy về mô hình liên quan tới đường đi trên DAG, bao gồm nhưng không giới hạn ở đồ thị, cấu trúc dữ liệu và chuỗi. Trên DAG tổng quát, việc duy trì nhanh thông tin đường đi là không thực tế, vì chỉ riêng mô tả một đường đi đã cần $\Omega(n)$ bit. Do đó bài viết tập trung vào các DAG có tổng số đường đi tương đối nhỏ; nhiều mô hình DAG thực dụng, chẳng hạn máy tự động hữu tố, thỏa tính chất này.

Bài viết chủ yếu giới thiệu ba thuật toán có thể duy trì thông tin đường đi trên DAG trong thời gian $O(\text{polylog } S)$, với S là tổng số đường đi: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Trong đó, phân rã chuỗi trên DAG có cài đặt gọn, thao tác sửa đổi tương đối dễ, nên thường thực dụng hơn trong OI. Cây cân bằng bền vững có độ phức tạp thời gian tối ưu và khả năng mở rộng mạnh, nhưng tốn bộ nhớ và khó cài đặt hơn. Độ phức tạp thời gian của phương pháp tách-trộn cây cân bằng vẫn còn mở; hiện chỉ có chứng minh cận trên $O(\log S \log q)$ mà chưa xây dựng được dữ liệu làm chặt cận này. Tuy vậy, vì quan hệ giữa cập nhật và truy vấn của nó ngược với hai thuật toán trước, tức truy vấn là offline còn DAG có thể bị bắt buộc đưa ra online, nó vẫn có một số ứng dụng khó thay thế.

Phần 1 trình bày mô hình cơ bản của duy trì thông tin đường đi trên DAG. Phần 2 lần lượt trình bày ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng. Phần 3 nêu các ứng dụng trong một số bối cảnh.

3.1 Mô hình cơ bản

Cho một DAG có n đỉnh, trong đó các cạnh ra của mỗi đỉnh có thứ tự. Sắp các đỉnh mà đỉnh i ($1 \leq i \leq n$) trở tới thành dãy G_i . Đỉnh i có trọng số nguyên dương c_i , và cạnh ra thứ j của nó có trọng số cạnh j .

Sắp tất cả các đường đi bắt đầu tại 1 và kết thúc ở một đỉnh có bậc ra bằng 0 theo thứ tự từ điển của dãy trọng số cạnh; cách sắp này là duy nhất. Ký hiệu dãy đường đi là P_i ($1 \leq i \leq S$), trong đó S là tổng số đường đi thỏa điều kiện.

Có q truy vấn. Mỗi truy vấn cho một số nguyên dương k , yêu cầu tính tổng trọng số đỉnh trên P_k .

Giả sử số cạnh của DAG không vượt quá $2n$. Dòng ràng buộc trong bản quét bị OCR nhiễu; các phần đọc được gồm $k \leq 10^{18}$ và các chặn đa thức thông thường cho n, q, c_i .

3.2 Lời giải

Vì $k \leq 10^{18}$, chỉ cần giữ lại 10^{18} đường đi đầu của DAG. Trên thực tế, có thể dùng DP $O(n)$ để tính số đường đi từ mỗi đỉnh tới các đỉnh bậc ra 0, đồng thời lấy min với 10^{18} , rồi chỉ giữ các đỉnh có giá trị DP nhỏ hơn 10^{18} . Ngoài ra cần sao chép các đỉnh nằm trên đường đi nhỏ thứ 10^{18} theo thứ tự từ điển và giữ với các cạnh ra của chúng một tiên tố trở tới nhóm đỉnh trước. Vì vậy chỉ cần xét trường hợp $S \leq 10^{18}$.

Bằng cách nhị phân hóa bậc ra, dễ thấy ta chỉ cần xét trường hợp bậc ra của mỗi đỉnh là 0 hoặc 2. Trong phân tích dưới đây giả sử $S > n$.

3.2.1 Phân rã chuỗi trên DAG

Nếu DAG là một cây có hướng, hoặc một rừng có hướng, tức bậc vào của mỗi đỉnh không vượt quá 1, hiển nhiên có thể dùng phân rã nặng–nhẹ để duy trì thông tin trên chuỗi.

Ta mở rộng cách làm này lên DAG tổng quát hơn. Với mỗi cạnh $u \rightarrow v$, xét số đường đi và số tiền tố đường đi liên quan tới hai đầu cạnh, rồi chọn cạnh nặng theo đỉnh kế tiếp và tiền nhiệm có giá trị lớn nhất. Bản quét nhận diện nhiều ở công thức định nghĩa chính xác của f_i, h_i, pre_i ; phần đọc được cho thấy f_i là số đường đi kết thúc ở i , h_i là số đường đi bắt đầu từ i , còn pre_i là tiền nhiệm nặng của i .

Cạnh $u \rightarrow v$ là cạnh nặng khi v là lựa chọn nặng của u và u là tiền nhiệm nặng của v . Dễ thấy các cạnh nặng tạo thành một số chuỗi rời nhau, gọi là chuỗi nặng.

Xét một đường đi bất kỳ trên DAG. Dọc đường đi đó, f_i giảm dần còn h_i tăng dần. Với mỗi cạnh nhẹ, ít nhất một trong hai đại lượng phải giảm hoặc tăng theo hệ số 2. Vì vậy trên một đường đi chỉ có $O(\log S)$ cạnh nhẹ.

Do đó, từ đỉnh 1 ta có thể mỗi lần tìm bằng nhị phân phần chung giữa đường đi được hỏi và chuỗi nặng hiện tại; trên mỗi chuỗi nặng tiền xử lý tổng tiền tố. Nhờ vậy bài toán ở trên được giải trong thời gian $O(n + q \log S \log n)$ và bộ nhớ $O(n)$. Nút thắt nằm ở phép nhị phân trong mảng f .

Tối ưu. Tham khảo kỹ thuật cây nhị phân cân bằng toàn cục trong phân rã cây, độ phức tạp có thể hạ xuống $O(n + q \log S)$. Với một chuỗi nặng cố định, đặt $w_i = f_i - f_{i+1}$, với quy ước $f_{s+1} = 0$. Dựng một cây nhị phân trên toàn bộ chuỗi sao cho trong một đoạn $[l, r]$, gốc con là vị trí đầu tiên có tổng trọng số hai phía đều không vượt quá một nửa tổng đoạn; nếu không tồn tại thì lấy đầu đoạn. Tiền xử lý LCA trên cây nhị phân này.

Nếu u là vị trí vào chuỗi nặng hiện tại và v là vị trí ra, ta tiền xử lý LCA giữa mỗi đỉnh với cuối chuỗi nặng rồi nhị phân từ $\text{LCA}(u, k)$ đi xuống. Độ phức tạp của phép nhị phân trên cây nhị phân chính là khoảng cách giữa $\text{LCA}(u, k)$ và v trên cây đó.

Bắt đầu từ v và đi lên hai bước mỗi lần, tổng trọng số của cây con ít nhất tăng gấp đôi, trừ tình huống lá biên. Sau một bước đi xuống từ LCA, tổng cây con không vượt quá đại lượng tương ứng ở điểm vào. Cộng trên mọi chuỗi nặng của một truy vấn, vì đại lượng của chuỗi hiện tại không nhỏ hơn chuỗi kế tiếp, tổng độ phức tạp nhị phân là $O(\log S)$.

Dùng splay hoặc treap cũng đạt cùng bậc độ phức tạp; bài viết không nhắc lại.

Mở rộng tối ưu. Dùng chia để trị, tức thông tin nửa nhóm trên đoạn với tiền xử lý $O(n \log n)$ và truy vấn $O(1)$ dựa trên hợp nhất hai đoạn, kết hợp DP, có thể hỗ trợ truy vấn giá trị lớn nhất của tổng trọng số đỉnh trên đoạn $P_l, P_{l+1}, \dots, P_r$ trong thời gian $O(n \log n + q \log S)$. Cụ thể, với mỗi đỉnh tính giá trị đường đi lớn nhất bắt đầu ở nó, rồi trên mỗi chuỗi nặng lưu cực trị đoạn của các thông tin đó.

Với các bài toán có sửa điểm đơn, sửa chuỗi đơn giản và truy vấn thông tin trên một chuỗi, cũng có thể tổng quát hóa cây đoạn có trọng số để giảm độ phức tạp mỗi thao tác xuống $O(\log S)$.

Bản quét chứa một công thức chọn gốc cây nhị phân phụ thuộc vào hai dãy $w_i = f_i - f_{i+1}$ và $x_i = h_i - h_{i-1}$, cùng tính chẵn lẻ của độ sâu d ; công thức này quá nhiều để chép chính xác. Ý chính đọc được là khoảng cách từ điểm vào tới LCA và từ điểm ra tới LCA đều bị chặn bởi các logarit của tỉ lệ tổng trọng số, nên cộng trên mọi chuỗi nặng vẫn không vượt quá $O(\log S)$.

Ngoài ra, với thông tin khả nghịch có sửa điểm đơn, xây cây đoạn có trọng số riêng theo hai phía cũng đạt truy vấn đơn $O(\log S)$. Tuy nhiên hằng số lớn; nếu thông tin cần duy trì đơn giản thì thuật toán này không có ưu thế rõ rệt so với BIT.

3.2.2 Cây cân bằng bền vững

Xét bài toán gốc theo thứ tự tô-pô đảo của DAG. Với mỗi đỉnh i , xét dãy Q_i gồm mọi đường đi bắt đầu từ i .

Rõ ràng, nếu bậc ra của i bằng 0 thì $Q_i = ((i))$. Nếu bậc ra của i bằng 2, với hai cạnh ra theo thứ tự, Q_i nhận được bằng cách ghép hai dãy đường đi của hai con, rồi chèn i vào đầu mỗi phần tử.

Ta chỉ duy trì tổng trọng số đỉnh của từng đường đi trong Q_i . Khi đó cần hỗ trợ các thao tác sau trong $O(\log S)$:

1. ghép hai dãy số nguyên độ dài không vượt quá S thành một dãy mới;
2. cộng cùng một hằng số vào mọi phần tử của một dãy;
3. cho k , hỏi phần tử thứ k của một dãy.

Dùng cây cân bằng để duy trì. Do có thao tác tự sao chép, ta cần một loại cây không có con trở cha và chiều cao chặt $O(\log S)$. Bài viết dùng WBLT, tức Weight Balanced Leafy Tree. Trong WBLT, mỗi lá biểu diễn một phần tử; mỗi nút trong có cây con trái và phải, và kích thước mỗi cây con không nhỏ hơn một hằng số cân bằng nhân với tổng kích thước. Mỗi nút có thể lưu thông tin đoạn. Ở đây, lá biểu diễn đường đi và mỗi nút giữ một nhãn cộng lười cho cây con.

Với thao tác ghép, chỉ cần merge hai WBLT. Khi ghép hai cây A, B và giả sử $|A| \geq |B|$, nếu tỉ lệ kích thước đã cân bằng thì tạo một gốc mới với hai cây con A, B . Nếu không, xét cây con phải của A ; khi ghép trực tiếp vẫn thỏa cân bằng thì gắn B vào phía phải. Nếu vẫn chưa thỏa, tiếp tục tách cấu trúc bên phải của A , ghép các phần tương ứng rồi hợp nhất trở lại. Bằng quy nạp, ghép hai WBLT có tỉ lệ kích thước là hằng số tổn $O(1)$; vì các đệ quy còn lại chỉ đi về một phía, tổng thời gian ghép không vượt quá $O(\log S)$. Trong bài này còn có thể lợi dụng tính khả trừ và giao hoán của thông tin để làm nhãn gần như vĩnh viễn, giảm hằng số số lần *pushdown*.

Thao tác cộng toàn dãy chỉ sửa nhãn ở gốc. Thao tác hỏi phần tử thứ k đệ quy từ gốc xuống lá. Vì vậy bài toán ban đầu được giải trong thời gian $O(n + q \log S)$ và bộ nhớ $O(n \log S)$.

Mở rộng của lời giải WBLT. Ngoài ghép, WBLT và một số cây cân bằng khác còn hỗ trợ tách: cho cây A và $k \in [0, |A|]$, trả về hai WBLT lần lượt biểu diễn k phần tử đầu và $|A| - k$ phần tử sau.

Cụ thể, đệ quy từ trên xuống sẽ tách A thành một số cây con có độ sâu tăng dần và một số cây con có độ sâu giảm dần; sau đó ghép chúng từ dưới lên để thu được hai cây kết quả. Vì chi phí ghép tỉ lệ với logarit tỉ lệ kích thước của hai phần, để chứng minh thao tác tách tổn $O(\log S)$.

Nói cách khác, ngoài ghép dãy, sửa toàn cục và hỏi điểm đơn, cây cân bằng bền vững còn hỗ trợ cắt lấy một đoạn dãy. Tất nhiên cũng có thể hỗ trợ sửa đoạn và hỏi đoạn; trong bài toán gốc, đoạn này tương ứng với một đoạn của dãy đường đi P .

Do cấu trúc phức tạp, cây cân bằng bền vững khó hỗ trợ sửa trọng số đỉnh của DAG.

Thảo luận liên quan. Có thể thấy mỗi DAG mà mọi đỉnh có bậc ra 0 hoặc 2 tương đương về cấu trúc với một cây Leafy Tree bền vững.

Điều này có nghĩa: khi không có tính chất đặc biệt và số thao tác ít, ta có thể duy trì trực tiếp DAG để cài ghép dãy và cắt dãy, bỏ qua các thao tác giữ cân bằng. Định nghĩa độ sâu dep_v của đỉnh v là số cạnh trên đường đi dài nhất bắt đầu ở v . Khi ghép A, B thành C , chỉ cần tạo một đỉnh mới có hai con A, B và đặt $\text{dep}_C = \max(\text{dep}_A, \text{dep}_B) + 1$. Khi tách A thành B, C , thời gian không vượt quá dep_A , vì trong quá trình đệ quy mỗi độ sâu chỉ bị truy cập ở trước và sau nhiều nhất một lần. Bằng cách ghép từ sau ra trước dọc ranh giới giữa B và C , ta đảm bảo $\max(\text{dep}_B, \text{dep}_C) \leq \text{dep}_A$. Sau $O(q)$ thao tác ghép và tách, độ sâu không vượt quá q , nên tổng độ phức tạp là $O(q^2)$. Đây cũng đạt cận dưới lý thuyết trong mô hình này, vì

sau q bước số đường đi có thể đạt $\exp(\Theta(q))$. Ưu thế của cây cân bằng bền vững nằm ở các trường hợp DAG có cấu trúc đặc biệt hơn.

Ngoài ra, cấu trúc phân rẽ chuỗi trên DAG kết hợp cây đoạn có trọng số ở phần trước cũng có thể được xem như một DAG “cân bằng hơn” nhưng tương đương với DAG gốc theo ý nghĩa dãy đường đi.

3.2.3 Tách-trộn cây cân bằng

Xét phiên bản offline của bài toán gốc: tại đỉnh 1, duy trì tập các chỉ số k xuất hiện trong mọi truy vấn.

Duyệt các đỉnh theo thứ tự tô-pô từ trước ra sau. Khi tới đỉnh i , mọi truy vấn đang nằm tại i được cộng thêm c_i vào đáp án. Nếu i có bậc ra 2, chuyển toàn bộ truy vấn này tới các đỉnh mà i trở tới; nửa sau cần trừ đi kích thước của nửa trước khỏi chỉ số k tương ứng.

Cần duy trì nhiều dãy không giảm và hỗ trợ ba thao tác sau; sau khi bị thao tác, dãy cũ biến mất:

1. tách một dãy thành hai dãy mới theo ngưỡng k ;
2. cộng k vào mọi phần tử của một dãy;
3. trộn hai dãy đã sắp xếp thành một dãy mới.

Dùng cây cân bằng để duy trì các dãy này thì hai thao tác đầu là trực tiếp.

Với thao tác trộn hai dãy A và B thành C , chia các vị trí $1, 2, \dots, |A| + |B|$ thành t đoạn liên tiếp cực đại sao cho các phần tử trong mỗi đoạn của C trước khi trộn đều thuộc cùng một dãy cũ. Dùng nhị phân từ trái sang phải hoặc đệ quy từ gốc xuống đều cho lời giải đơn giản $O(t \log q)$.

Tiếp theo chứng minh tổng t không vượt quá $O((n + q) \log S)$. Với một dãy A , định nghĩa thế năng

$$\Phi(A) = \sum_i \log(2 + A_{i+1} - A_i).$$

Tổng thế năng luôn không âm và không vượt quá $O(q \log S)$. Thao tác tách làm thế năng thay đổi nhiều nhất $O(\log S)$, thao tác cộng toàn dãy không đổi thế năng. Vì vậy chỉ cần chứng minh trong một thao tác trộn, thế năng giảm ít nhất $\Omega(t) - O(\log S)$.

Bản quét của chuỗi bất đẳng thức chi tiết bị nhiễu ở chỉ số, nhưng lập luận đọc được như sau: gọi các đoạn cực đại sau trộn là $[l_i, r_i]$ và đặt $d_i = l_i - r_{i-1}$ giữa hai đoạn kề nhau. Khi ghép hai đoạn kề, số hạng logarit tương ứng được thay bằng logarit của khoảng cách lớn hơn; dùng $\min(d_i, d_{i+1})$ và $\max(d_i, d_{i+1})$ để chặn, mỗi lần đối đoạn đóng góp một lượng giảm hằng số, còn hai biên chỉ tạo sai số $O(\log S)$. Do đó tổng số đoạn cực đại trên mọi lần trộn bị chặn bởi tổng thế năng tăng.

Ta thu được lời giải thời gian $O((n + q) \log S \log q)$ và bộ nhớ $O(n + q)$.

3.2.4 Mở rộng và thảo luận

Nhìn lại bốn thao tác mà cây cân bằng bền vững ở phần trên hỗ trợ: ghép dãy, cắt dãy, cộng toàn cục và hỏi điểm đơn. Nếu xem các thao tác này như một DAG, rồi duyệt ngược và dùng cây cân bằng để duy trì mọi truy vấn, nút thất vẫn là tách cây, cộng toàn cục và trộn cây. Nói cách khác, vấn đề mà cây cân bằng bền vững và vấn đề mà tách-trộn cây cân bằng giải quyết phần nào là chuyển vị của nhau.

Đáng tiếc là cận trên tốt nhất đã biết của tách-trộn cây cân bằng là $O(\log S \log q)$, dù cũng chưa có dữ liệu đạt chặt cận này, còn cây cân bằng bền vững đạt $O(\log S)$. Trong các bài toán liên quan tới bài viết này, thuật toán tách-trộn chỉ hơn cây cân bằng bền vững khi có yêu cầu đặc biệt về bộ nhớ, hoặc khi một số thông tin trong DAG thao tác bị bắt buộc đưa ra online, chẳng hạn khi thao tác tách không phải tách theo “ $\geq k$ ” và “ $< k$ ” mà là tách theo k phần tử đầu và phần còn lại.

3.3 Ứng dụng

3.3.1 Duy trì quá trình thao tác trên dãy

Theo phần 2.2.2, xem DAG như một Leafy Tree bền vững giúp cài đặt thuận tiện các thao tác cắt dãy và ghép dãy.

Ví dụ 1. Có n dãy. Ban đầu dãy thứ i chỉ chứa một số.

Cần thực hiện q thao tác, mỗi thao tác thuộc một trong ba loại:

1. tạo một dãy mới bằng cách ghép hai dãy đã có;
2. tạo hai dãy mới, lần lượt là k phần tử đầu của một dãy đã có và các phần tử còn lại, trong đó k là số nguyên lớn không vượt quá độ dài dãy;
3. xuất số nghịch thế của một dãy modulo 998244353.

Ràng buộc đọc được: $1 < n < 100, 1 < q < 600$.

Lời giải ví dụ 1. Nếu không có thao tác loại 2, hiển nhiên có thể duy trì số lần xuất hiện của mỗi giá trị trong từng dãy và tổng số nghịch thế, mỗi thao tác tốn $O(n)$.

Dùng cách làm ở phần 2.2.2: với thao tác loại 2, ta đệ quy từ trên xuống để tách rồi ghép dần từ dưới lên, trong quá trình duy trì kích thước mỗi nút. Khi đó mỗi thao tác tách có thể viết lại thành $O(q)$ thao tác ghép, còn mỗi thao tác ghép tốn $O(n^2 + n\omega)$, trong đó ω là số bit của kích thước, thường xem là 32 hoặc 64. Tổng độ phức tạp là $O(n^2q^2 + n\omega q^2)$.

3.3.2 Ứng dụng trên máy tự động hậu tố

Trong lý thuyết chuỗi của OI, máy tự động hậu tố (SAM) là một thuật toán hiệu quả và thực dụng.

Cụ thể, với chuỗi S trên bảng chữ cái Σ , xét tập vị trí kết thúc của xâu con T trong S , ký hiệu là R_T . Với các T khác nhau, quan hệ giữa các R_T chỉ có thể là bao hàm hoặc rời nhau; vì vậy chỉ có $O(|S|)$ tập R_T khác nhau về bản chất và chúng tạo thành một cấu trúc cây, thực chất là cây hậu tố của chuỗi đảo sau khi nén các đỉnh bậc hai.

Định nghĩa tự động A có tập đỉnh là các R_T , với T là xâu con của S , và có cạnh ký tự a từ R_T tới R_{Ta} nếu Ta cũng là xâu con của S . Từ các tính chất của SAM, A có kích thước $O(|S|)$ và có thể xây dựng cây cùng tự động trong $O(|S|)$.

Các xâu con khác nhau của S tương ứng một-một với các đường đi bắt đầu từ xâu rỗng trong A . Nhờ đó, thông tin về xâu con khác nhau được chuyển thành thông tin đường đi trên tự động, giúp xử lý hiệu quả một số bài toán khá khó nếu chỉ nhìn từ góc độ cây hậu tố.

Ví dụ 2. Nguồn bài là UOJ Long Round #1, bài D¹, đã lược bỏ chi tiết không cần thiết.

Cho chuỗi S và mảng c_i . Với xâu con T của S , ký hiệu tập vị trí kết thúc của T trong S là R_T và định nghĩa $f(T) = |R_T| \sum_{i \in R_T} c_i$.

Có q truy vấn, mỗi truy vấn yêu cầu tính tổng $f(T)$ trên một họ xâu con liên tiếp theo mô tả của đề gốc. Ràng buộc đọc được: $|S| \leq 5 \cdot 10^5$, bảng chữ cái 26. Một phần dòng truy vấn trong bản quét bị OCR nhiễu nên không chép lại chính xác.

¹<https://uoj.ac/problem/577>

Lời giải ví dụ 2. Xây cây hậu tố của chuỗi đảo và SAM của S . Trên mỗi đỉnh của SAM, giá trị f là cố định, còn truy vấn đúng là tính tổng f trên một đường đi của SAM.

Dùng phân rã chuỗi trên DAG và duy trì tổng tiền tố trên mỗi chuỗi nặng là giải được. Nút thắt là tìm độ dài phần chung giữa đường đi truy vấn và chuỗi nặng hiện tại; dùng bảng thưa LCP với tiền xử lý $O(|S| \log |S|)$ và truy vấn $O(1)$.

Thảo luận ví dụ 2. Thực ra SAM chỉ có $O(|S|)$ đường đi tối đại, tương ứng với mọi hậu tố của S . Dựa trên tính chất này, tồn tại thuật toán đơn giản hơn.

Khi trải mọi đường đi của SAM ra, ta thu được một cây chỉ có $O(|S|)$ lá, thực chất là cây hậu tố của chuỗi thuận. Mỗi truy vấn tương đương tính tổng trọng số đỉnh trên một chuỗi của cây này; mặt khác các đỉnh này đều tương ứng với đỉnh SAM.

Nén các đỉnh bậc hai của cây này và xét thao tác nén trên tự động: xóa mọi đỉnh có bậc ra 1 trên tự động, với mọi cạnh trở tới chúng thì đổi đích cạnh thành đỉnh mà đích cũ trở tới, lặp đến khi đích có bậc ra khác 1, đồng thời đổi ký tự trên cạnh thành một chuỗi. Bằng cách lưu mảng trên cạnh, tự động sau nén vẫn duy trì được tổng đoạn; khi trải tự động ra sẽ trực tiếp thu được một cây hậu tố chuỗi thuận đã nén có $O(|S|)$ đỉnh. Xử lý tổng tiền tố trực tiếp trên cây này là đủ. Tổng độ phức tạp là $O(|S| + q)$.

Đáng chú ý, tập đỉnh của SAM nén của chuỗi thuận và chuỗi đảo là giống nhau về bản chất. Điều này đem lại một góc nhìn mới cho các bài toán xâu con liên quan tới cả đầu trái và đầu phải. Tuy nhiên thuật toán này cũng có hạn chế: cấu trúc SAM nén phụ thuộc vào tính đối xứng giữa chuỗi thuận và chuỗi đảo, nên khả năng mở rộng kém hơn phân rã chuỗi trên DAG.

Ví dụ 3. Cho một trie S có n đỉnh. Định nghĩa xâu con của S là chuỗi tương ứng với một đường đi từ trên xuống.

Có q truy vấn. Mỗi truy vấn cho k , yêu cầu tìm xâu con khác nhau nhỏ thứ k của S hoặc báo không tồn tại. Để tránh đầu ra quá lớn, chỉ cần xuất tổng mã ASCII của mọi ký tự trong xâu đó.

Ràng buộc đọc được: $n, q \leq 10^5$, $k \leq 10^{18}$, bảng chữ cái 26.

Lời giải ví dụ 3. Cần xây SAM tổng quát. Ở đây S được định nghĩa là trie tạo bởi các chuỗi trên đường đi từ mọi đỉnh về gốc, còn tập *right* của SAM tương ứng một-một với đỉnh của trie.

Phân tích độ phức tạp của thuật toán xây SAM trên chuỗi không còn áp dụng trực tiếp, và số cạnh của SAM lúc này có thể đạt $n|\Sigma|$. Theo các tài liệu được trích dẫn, BFS trực tiếp trên trie rồi dùng thuật toán SAM cho chuỗi là tuyến tính theo kích thước kết quả, nhưng tác giả không biết chứng minh đầy đủ. Cũng có thể dùng suffix array tổng quát bằng nhân đôi để tính cây, rồi từ đó tính SAM, tổng độ phức tạp $O(n \log n + n|\Sigma|)$; nếu dùng mở rộng của DC3 trên cây có thể bỏ thừa số log.

Sau khi xây được SAM, bài toán trở thành mô hình kinh điển của phần 1, và cả ba thuật toán ở phần 2 đều giải được. Chẳng hạn dùng phân rã chuỗi trên DAG thì tổng thời gian là $O(n \log n + n|\Sigma| + q \log^2 n)$, hoặc với tối ưu là $O(n \log n + n|\Sigma| + q \log n)$.

Ví dụ 4. Nguồn bài là UOJ tuyển chọn đội, bài C^2 .

Cho chuỗi S độ dài n và m xâu con của S làm mẫu. Có q truy vấn; mỗi truy vấn yêu cầu tổng số lần xuất hiện của mọi xâu mẫu trong một xâu con của S . Ràng buộc đọc được: $n \leq 4 \cdot 10^5$, $m \leq 10^5$, $q \leq 10^5$, bảng chữ cái 26.

²<https://uoj.ac/problem/697>

Lời giải ví dụ 4. Xây SAM của S và cây hậu tố của chuỗi đảo, rồi phân rã chuỗi trên DAG của SAM.

Với một xâu con T của S , số xâu mẫu trong các hậu tố của T được quyết định bởi đỉnh tương ứng với T trên SAM và các xâu mẫu tương ứng. Hơn nữa, chỉ cần lấy số xâu mẫu trên đường đi từ đỉnh đó về gốc trong cây hậu tố đảo, rồi trừ đi số xâu mẫu có độ dài lớn hơn $|T|$.

Với mỗi truy vấn, định vị xâu con thành một đường đi trên tự động và tính tổng đóng góp của mọi đỉnh trên đường đi đó. Định vị đường đi có thể làm trong $O((n+q)\log n)$ bằng cách tiền xử lý $O(n\log n)$ trên chuỗi gốc và các chuỗi nặng của DAG rồi hỏi LCP xâu con trong $O(1)$.

Với thống kê đáp án: loại đóng góp thứ nhất chỉ cần lưu tổng tiền tố của w_v trên mỗi chuỗi nặng; loại thứ hai chuyển thành bài toán đếm điểm trong hình bình hành nghiêng 45° trên từng chuỗi nặng, sau đổi tọa độ là bài toán đếm điểm hai chiều thông thường. Tùy cách cài LCP và đếm điểm, tổng độ phức tạp là $O((n+m)\log n + q\log^2 n)$ hoặc $O((n+m+q)\log n)$.

3.3.3 Thuật toán Euclid vạn năng

Trên lưới có đường thẳng $y = \frac{a}{b}x$ với $a, b \in \mathbb{N}^+$. Xét một điểm động trên đường thẳng này di chuyển theo hướng $x+y$ tăng trên một đoạn. Mỗi khi đi qua một đường ngang, ghi ký hiệu A ; mỗi khi đi qua một đường dọc, ghi ký hiệu B ; nếu đi qua điểm lưới thì ghi A trước rồi B sau. Cuối cùng thu được một dãy trên $\{A, B\}$, ví dụ với $a=3, b=2$ thì dãy là $ABBAB$.

Thay A và B bằng hai phần tử của một nửa nhóm, tức thông tin ghép được trong $O(1)$, mục tiêu của thuật toán Euclid vạn năng là tính phần tử nửa nhóm ứng với toàn bộ dãy.

Trước hết giả sử đoạn di chuyển là từ $(0,0)$ tới (a,b) và $\gcd(a,b)=1$. Nếu $a > b$, đặt $k = \lfloor a/b \rfloor$. Dễ thấy sau mỗi A đều có k ký hiệu B . Thay mọi đoạn AB^k trong dãy gốc bằng một ký hiệu mới tương ứng; về mặt hình học, điều này tương đương tác động một phép biến đổi tọa độ lên mặt phẳng và đường thẳng, đưa bài toán về kích thước nhỏ hơn. Lặp quá trình $O(\log a)$ lần sẽ giải được bài toán.

Dùng DAG để mô tả thuật toán trên: ban đầu có hai đỉnh A, B . Mỗi lần tạo một đỉnh C , nối cạnh tới A và B theo phép thay thế tương ứng, rồi thay (a,b) bằng $(a-b, b)$ hoặc $(a, b-a)$ tùy bên lớn hơn, cho tới khi $a=b$.

Số cạnh của DAG có thể đạt độ phức tạp của phép trừ lặp, tức $O(a)$. Chỉ cần dùng nhân đôi để tối ưu các thao tác trừ liên tiếp là có thể giảm kích thước đồ thị xuống $O(\log(a+b))$.

Dãy AB do quá trình Euclid vạn năng sinh ra chính là dãy các đỉnh cuối của mọi đường đi khi trải DAG theo thứ tự DFS. Vì vậy phiên bản tổng quát của thuật toán Euclid vạn năng tương đương truy vấn đoạn trên Leafy Tree bền vững ứng với DAG này. Nhiều bài toán Euclid cổ điển, như tổng $\lfloor ai/b \rfloor$ hoặc $\min_i (ai \bmod b)$, có thể chuyển thành mô hình này; hơn nữa DAG này cho phép giải một số bài toán phức tạp hơn liên quan tới quá trình Euclid.

Ví dụ 5. Đây là bài mẫu thuật toán Euclid vạn năng trên LOJ³.

Cho n, m, l và hai ma trận vuông A, B kích thước không vượt quá 10. Cần tính biểu thức ma trận theo dãy Euclid do đề bài định nghĩa. Bản quét của công thức mục tiêu bị nhiễu nên không chép lại an toàn. Ràng buộc đọc được: $n, m, l \leq 10^{18}$.

Lời giải ví dụ 5. Xét dãy AB ứng với đường thẳng liên quan trong đề. Duy trì ma trận M , ban đầu $M = I$. Từ vị trí thứ hai của dãy trở đi, mỗi khi gặp A thì đặt $M = MB$; mỗi khi gặp B thì cộng M vào đáp án rồi đặt $M = AM$. Khi kết thúc, đáp án là giá trị cần tìm.

Tác động của một đoạn của dãy có thể mô tả bằng một bộ ma trận: nếu trước khi đi vào đoạn có

³<https://loj.ac/p/6440>

trạng thái M , sau đoạn đáp án được cộng thêm một biểu thức tuyến tính theo M và trạng thái mới cũng là một biến đổi tuyến tính của M . Vì vậy thông tin đoạn ghép được trong $O(1)$, và có thể áp dụng trực tiếp phiên bản truy vấn đoạn của thuật toán Euclid vạn năng.

Ví dụ 6. Nguồn bài là CTT 2021 Day 4 T3, đã lược bỏ chi tiết không cần thiết.

Xét đồ thị vô hướng n đỉnh, trong đó i nối với $(i + d) \bmod n$. Trọng số đỉnh lấy theo một dãy tuần hoàn độ dài m : $w_i = a_{i \bmod m}$.

Cần thực hiện q lần sửa điểm đơn trên trọng số, sau mỗi lần sửa tính kích thước tập độc lập có trọng số lớn nhất của đồ thị.

Ràng buộc đọc được: $d < n \leq 10^9$, $m \leq 2 \cdot 10^5$, $w_i \leq 400$, $q \leq 10^5$.

Lời giải ví dụ 6. Dễ thấy đồ thị gồm $\gcd(n, d)$ chu trình, và chỉ có một số loại chu trình khác nhau về bản chất. Sau khi xóa mọi đỉnh từng bị sửa ít nhất một lần, đồ thị còn lại gồm $O(q)$ chuỗi và một số loại chu trình. Chỉ cần tiền xử lý đáp án của mỗi chuỗi và mỗi loại chu trình rồi cộng lại; với mỗi chuỗi, dùng một ma trận 2×2 biểu diễn trạng thái chọn hoặc không chọn hai đầu để tính tập độc lập có trọng số lớn nhất, rồi duy trì cây đoạn trên các điểm quan trọng.

Xét thông tin trên một chu trình. Trên lưới, vẽ đoạn $y = \frac{d}{n}x$ với $x \in [0, n)$ và xây dãy cùng DAG tương ứng của thuật toán Euclid vạn năng. Đồng thời duy trì một giá trị x ; các giá trị x khác nhau tương ứng với các chu trình khác nhau. Duyệt dãy và duy trì ma trận 2×2 ; mỗi khi gặp một loại ký hiệu thì cập nhật x theo modulo m , đồng thời ghép thông tin tương ứng với w_x . Sau khi thao tác xong, ma trận nhận được là đáp án của chu trình; nếu chỉ thao tác trên một đoạn thì nhận được đáp án của chuỗi.

Thông tin đoạn của dãy có thể định nghĩa là m ma trận 2×2 , biểu diễn tác động khi giá trị ban đầu của x lần lượt là $0, 1, \dots, m - 1$, cùng một độ dời cho x sau khi đi qua đoạn. DP trên DAG Euclid tính được mọi thông tin đoạn trong $O(m \log n)$, từ đó lập tức có đáp án chu trình.

Đáp án chuỗi là truy vấn đoạn trên thông tin đường đi của DAG. Vì DAG này chỉ có $O(\log n)$ đỉnh, DFS trực tiếp từ trên xuống đạt $O(q \log n)$. Tổng độ phức tạp là $O((m + q) \log n)$.

Kết luận

Bài viết đã giới thiệu ba thuật toán: phân rã chuỗi trên DAG, cây cân bằng bền vững và tách-trộn cây cân bằng, đồng thời đưa ra ứng dụng của chúng trong một số bàiOI. Tác giả phỏng đoán phía sau mô hình duy trì đường đi trên DAG có thể còn những tính chất đẹp và sâu hơn; các mở rộng khác của ba thuật toán và mối liên hệ giữa chúng vẫn cần được khai thác thêm. Hy vọng bài viết có thể gợi mở hướng nghiên cứu tiếp theo cho độc giả quan tâm.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn huấn luyện viên Yang Yuliang của đội tuyển quốc gia đã hướng dẫn; cảm ơn cha mẹ, nhà trường, các thầy cô và bạn học đã bồi dưỡng, giảng dạy và hỗ trợ; cảm ơn các anh chị và bạn bè đã trao đổi, góp ý, truyền cảm hứng và đọc bản thảo; cảm ơn mọi nguồn tư liệu tham khảo; và cảm ơn độc giả đã dành thời gian đọc bài viết.

Tài liệu tham khảo

1. Zhang Zheyu. *Bàn về thuật toán chia để trị trên cây*. Luận văn đội tuyển quốc gia IOI 2019, 2019.

2. Wang Siqi. *Leafy Tree và cây cân bằng trọng số được cài đặt từ nó*. Luận văn đội tuyển quốc gia IOI 2018, 2018.
3. SF. *Bàn về máy tự động hậu tố nén*. Luận văn đội tuyển quốc gia IOI 2020, 2020.
4. Liu Yanyi. *Mở rộng máy tự động hậu tố trên trie*. Luận văn đội tuyển quốc gia IOI 2015, 2015.
5. J. A. Blumer. *Algorithms for the directed acyclic word graph and related structures*. Ph.D. Thesis, Denver University, 1985.
6. A. Blumer, J. Blumer, D. Haussler, R. M. McConnell, A. Ehrenfeucht. *Complete inverted files for efficient text retrieval and analysis*. J. ACM 34(3)(1987), pp. 578–595.
7. J. Karkkainen, P. Sanders, S. Burkhardt. *Linear work suffix array construction*. J. ACM 53(6)(2006), pp. 918–936.

4 Chủ đề chính

Từ mục lục đã đọc được trên ảnh render, tập năm 2022 bao phủ các nhóm chủ đề sau:

- duy trì thông tin đường đi trên DAG, DFT và các biến thể tổng quát;
- tư tưởng tham lam dựa trên so sánh và thuật toán xấp xỉ;
- đồ thị phẳng, tô màu đồ thị và các bài toán tương tác dạng tìm tham số;
- tối ưu không gian, ma trận Monge và phân tích bảng băm;
- đại số tuyến tính, lý thuyết Lyndon, tổ hợp chuỗi và mô phỏng luồng chi phí.

5 Ghi chú về trích xuất

pdftotext không trích xuất được văn bản có nghĩa từ tập 2022; kết quả kiểm tra các trang đầu chỉ gồm dấu sang trang. Mục lục và bài đầu tiên trong bản dịch này được đọc bằng cách render bằng pdftoppm rồi OCR bằng tesseract với ngôn ngữ chi_sim+eng. Một số công thức ở các mục 2.1, 2.3, 3.2 và 3.3 bị nhiễu trên ảnh/OCR; các chỗ đó được đánh dấu bằng ghi chú OCR tiếng Việt thay vì tự suy diễn công thức.